hopfield-lean: Formal Proofs of Hopfield Network Energy Descent and Attractor Convergence in Lean 4

Ben Cassie
2026

Abstract

`hopfield-lean` is a Lean 4 / Mathlib library formalising the classical energy-descent and attractor-convergence argument for discrete Hopfield networks. The library models finite binary states as maps `Fin N -> ℝ`, proves the master single-unit energy-change identity, verifies non-increase and strict decrease of the Hopfield energy under asynchronous updates, characterises fixed points by local energy minimality, and proves that no infinite sequence of non-trivial asynchronous updates exists. The development contains zero `sorry`, zero `admit`, and no project-specific axioms. Its significance is twofold: it provides a machine-checked reference for a foundational result in associative memory, and it supplies a reusable Lean artifact for future formal work on attractor networks, neural computation, optimisation, and AI safety.

1. Introduction

Hopfield networks are one of the canonical models of associative memory. Introduced by Hopfield in 1982, they describe a finite network of binary units whose state evolves through local asynchronous updates. Each unit responds to a local field determined by the other units, a symmetric weight matrix, and a threshold vector. The network is equipped with an energy function, and the classical theorem states that asynchronous updates never increase this energy. When an update actually changes the state, the energy strictly decreases. Since the state space is finite, the process cannot continue changing state forever, and therefore reaches a fixed-point attractor.

This result is often presented as an elementary argument from neural computation and statistical physics. However, several hypotheses are essential: the weight matrix must be symmetric, the diagonal weights must vanish, the update rule must handle ties carefully, and convergence depends on finiteness of the binary state space rather than on continuous-time Lyapunov or ODE machinery. These details are easy to gloss over in informal mathematics, but they become explicit obligations in a proof assistant.

`hopfield-lean` formalises this convergence spine in Lean 4 / Mathlib. It defines finite Hopfield states, the Hopfield energy, local fields, asynchronous updates, single-unit flips, fixed points, and a Boolean encoding of valid states. It then proves the core energy identities and the finite-state convergence theorem. The result is not a new Hopfield theorem. It is a machine-checked, importable formalisation of the standard theorem under precise hypotheses.

The library is part of a broader programme of Lean 4 formalisation for learning and dynamical systems. Related repositories formalise gradient descent convergence, Kuramoto synchronisation, contraction theory, Lotka-Volterra conservation, and accelerated optimisation. In that context, `hopfield-lean` contributes the attractor-network component: a verified finite-state Lyapunov argument for associative memory.

2. Library Overview

The project is organised into five Lean modules plus a root import file:

- `Hopfield/Defs.lean` defines valid states, energy, local fields, asynchronous updates, single-unit flips, fixed points, and state encodings.
- `Hopfield/ValidState.lean` proves arithmetic and update lemmas for `{±1}` states.
- `Hopfield/Energy.lean` proves linear and quadratic energy-difference identities, then derives energy descent and strict descent.
- `Hopfield/FixedPoint.lean` proves the energy change under a flip and the fixed-point characterisation.
- `Hopfield/Convergence.lean` proves finite-state convergence: there is no infinite sequence of non-trivial asynchronous updates.
- `Hopfield.lean` is the root module importing the library.

The project depends on:

- Lean `v4.28.0`
- Mathlib `v4.28.0`

The formal setting is a finite index type `Fin N`. A Hopfield state is represented as a real-valued function:

```
Fin N -> ℝ
```

A state is valid when each coordinate is either `1` or `-1`:

```
def ValidState (s : Fin N -> ℝ) : Prop :=
  ∀ i, s i = 1 ∨ s i = -1
```

The energy function is:

```
E(s) = -(1/2) * sum_i sum_j s_i W_ij s_j + sum_i theta_i s_i.
```

In Lean:

```
noncomputable def energy (W : Matrix (Fin N) (Fin N) ℝ) (θ : Fin N -> ℝ)
    (s : Fin N -> ℝ) : ℝ :=
  -(1 / 2 : ℝ) * ∑ i : Fin N, ∑ j : Fin N, s i * W i j * s j +
  ∑ i : Fin N, θ i * s i
```

The local field at unit `i` is:

```
h_i(s) = sum_j W_ij s_j - theta_i.
```

The asynchronous update changes only coordinate `i`: it sets the coordinate to `1` if the local field is positive, to `-1` if the local field is negative, and preserves the current value on a tie. This tie convention is important: it makes zero local field a no-op and supports the fixed-point characterisation.

The main structural assumptions on the weight matrix are:

- symmetry: `W.IsSymm`
- zero diagonal: `∀ k, W k k = 0`

These are exactly the classical Hopfield hypotheses needed for the energy function to be a Lyapunov function for asynchronous updates.

The repository is available at:

https://github.com/velvetmonkey/hopfield-lean

3. Theorem Inventory

The source contains seven headline energy, fixed-point, and convergence results, supported by state-arithmetic and update lemmas.

Layer 1 - Valid State Mechanics

1. `ValidState.sq_eq_one` — For a valid state, every coordinate squares to one:

```
lemma ValidState.sq_eq_one {s : Fin N -> ℝ} (hs : ValidState s) (i : Fin N) :
    s i * s i = 1
```

2. `ValidState.neg_mem` — Negating a valid coordinate again gives a value in `{1, -1}`:

```
lemma ValidState.neg_mem {s : Fin N -> ℝ} (hs : ValidState s) (i : Fin N) :
    -(s i) = 1 ∨ -(s i) = -1
```

3. `ValidState.ne_zero` — Valid coordinates are nonzero:

```
lemma ValidState.ne_zero {s : Fin N -> ℝ} (hs : ValidState s) (i : Fin N) :
    s i ≠ 0
```

4. `ValidState.abs_eq_one` — Valid coordinates have absolute value one:

```
lemma ValidState.abs_eq_one {s : Fin N -> ℝ} (hs : ValidState s) (i : Fin N) :
    |s i| = 1
```

5. `asyncUpdate_valid` — Asynchronous updates preserve validity:

```
lemma asyncUpdate_valid {W : Matrix (Fin N) (Fin N) ℝ} {θ : Fin N -> ℝ}
    {s : Fin N -> ℝ} (hs : ValidState s) (i : Fin N) :
    ValidState (asyncUpdate W θ s i)
```

6. `flipUnit_valid` — Single-unit flips preserve validity:

```
lemma flipUnit_valid {s : Fin N -> ℝ} (hs : ValidState s) (i : Fin N) :
    ValidState (flipUnit s i)
```

7. `asyncUpdate_self` — The updated coordinate follows the threshold rule:

```
lemma asyncUpdate_self (W : Matrix (Fin N) (Fin N) ℝ) (θ : Fin N -> ℝ)
    (s : Fin N -> ℝ) (i : Fin N) :
    asyncUpdate W θ s i i =
      if 0 < localField W θ s i then 1
      else if localField W θ s i < 0 then -1
      else s i
```

8. `asyncUpdate_ne` — Coordinates other than the updated coordinate are unchanged:

```
lemma asyncUpdate_ne (W : Matrix (Fin N) (Fin N) ℝ) (θ : Fin N -> ℝ)
    (s : Fin N -> ℝ) {i j : Fin N} (hij : j ≠ i) :
    asyncUpdate W θ s i j = s j
```

9. `flipUnit_self` and `flipUnit_ne` — The flipped coordinate is negated and all others are unchanged.
10. `encodeState_injective_on_valid` — The Boolean encoding of valid states is injective on valid states:

```
lemma encodeState_injective_on_valid {s t : Fin N -> ℝ}
    (hs : ValidState s) (ht : ValidState t)
    (h : encodeState s = encodeState t) : s = t
```

This lemma is the bridge from real-valued {±1} vectors to finite Boolean state-space reasoning.

Layer 2 - Energy Identities and Descent

11. `linear_diff` — If only coordinate `i` changes, the linear threshold term changes by `θ_i(s'_i - s_i)`:

```
lemma linear_diff (θ : Fin N -> ℝ) (s s' : Fin N -> ℝ) (i : Fin N)
    (heq : ∀ j, j ≠ i -> s' j = s j) :
    (∑ j, θ j * s' j) - (∑ j, θ j * s j) = θ i * (s' i - s i)
```

12. `quadratic_diff` — Under symmetry and zero diagonal, if only coordinate `i` changes, the quadratic form changes by twice the local coupling term:

```
lemma quadratic_diff (W : Matrix (Fin N) (Fin N) ℝ) (s s' : Fin N -> ℝ)
    (i : Fin N) (hW_symm : W.IsSymm) (hW_diag : ∀ k, W k k = 0)
    (heq : ∀ j, j ≠ i -> s' j = s j) :
    (∑ a, ∑ b, s' a * W a b * s' b) - (∑ a, ∑ b, s a * W a b * s b) =
    2 * (s' i - s i) * ∑ j, W i j * s j
```

13. `energy_diff` — The master energy-change identity:

```
theorem energy_diff (W : Matrix (Fin N) (Fin N) ℝ) (θ : Fin N -> ℝ)
    (s s' : Fin N -> ℝ) (i : Fin N)
    (hW_symm : W.IsSymm) (hW_diag : ∀ k, W k k = 0)
    (heq : ∀ j, j ≠ i -> s' j = s j) :
    energy W θ s' - energy W θ s = -(s' i - s i) * localField W θ s i
```

Mathematically, this states:

```
E(s') - E(s) = -(s'_i - s_i) h_i(s).
```

14. `energy_descent` — A single asynchronous update never increases energy:

```
theorem energy_descent (W : Matrix (Fin N) (Fin N) ℝ) (θ : Fin N -> ℝ)
    (s : Fin N -> ℝ) (hW_symm : W.IsSymm) (hW_diag : ∀ k, W k k = 0)
    (hs : ValidState s) (i : Fin N) :
    energy W θ (asyncUpdate W θ s i) ≤ energy W θ s
```

15. `energy_strict_descent` — If an asynchronous update changes the state, the energy strictly decreases:

```
theorem energy_strict_descent (W : Matrix (Fin N) (Fin N) ℝ) (θ : Fin N -> ℝ)
    (s : Fin N -> ℝ) (hW_symm : W.IsSymm) (hW_diag : ∀ k, W k k = 0)
    (hs : ValidState s) (i : Fin N) (hne : asyncUpdate W θ s i ≠ s) :
    energy W θ (asyncUpdate W θ s i) < energy W θ s
```

These two theorems are the Lyapunov core of the library.

Layer 3 - Fixed Points and Attractor Convergence

16. `energy_flip_eq` — The energy change from flipping unit `i` is `2 s_i h_i`:

```
lemma energy_flip_eq (W : Matrix (Fin N) (Fin N) ℝ) (θ : Fin N -> ℝ)
    (s : Fin N -> ℝ) (hW_symm : W.IsSymm) (hW_diag : ∀ k, W k k = 0)
    (i : Fin N) :
    energy W θ (flipUnit s i) - energy W θ s =
    2 * s i * localField W θ s i
```

17. `fixed_point_iff` — A valid state is a fixed point iff no single-unit flip lowers energy:

```
theorem fixed_point_iff (W : Matrix (Fin N) (Fin N) ℝ) (θ : Fin N -> ℝ)
    (s : Fin N -> ℝ) (hW_symm : W.IsSymm) (hW_diag : ∀ k, W k k = 0)
    (hs : ValidState s) :
    IsFixedPoint W θ s ↔ ∀ i, energy W θ s ≤ energy W θ (flipUnit s i)
```

18. `convergence` — There is no infinite sequence of state-changing asynchronous updates:

```
theorem convergence (W : Matrix (Fin N) (Fin N) ℝ) (θ : Fin N -> ℝ)
    (hW_symm : W.IsSymm) (hW_diag : ∀ k, W k k = 0)
    (f : ℕ -> Fin N -> ℝ) (units : ℕ -> Fin N)
    (hvalid : ∀ n, ValidState (f n))
    (hstep : ∀ n, f (n + 1) = asyncUpdate W θ (f n) (units n))
    (hchange : ∀ n, f (n + 1) ≠ f n) : False
```

Informally, if every step is an asynchronous update and every step changes the state, then the assumptions are contradictory. The proof combines strict energy descent with finiteness of the valid state space.

4. Key Technical Highlights

Master Energy-Difference Identity

The central technical result is `energy_diff`:

```
E(s') - E(s) = -(s'_i - s_i) h_i(s).
```

This identity is the Hopfield Lyapunov argument in its most reusable form. It applies to any pair of states `s` and `s'` that differ only at coordinate `i`. The proof decomposes the energy into its quadratic term and linear threshold term.

The linear component is handled by `linear_diff`: because all coordinates except `i` are equal, all terms cancel except the `i`-th threshold term. The quadratic component is handled by `quadratic_diff`: symmetry lets the row and column contributions match, while the zero-diagonal hypothesis removes the self-interaction term. These two facts combine algebraically into the local-field expression.

This is the point where the classical Hopfield hypotheses become formal obligations. Without symmetry or the zero diagonal, the clean identity does not have the same form.

Energy Descent and Strict Descent

The theorem `energy_descent` instantiates `energy_diff` with `s' = asyncUpdate W θ s i`. The asynchronous update rule is chosen to align the sign of `(s'_i - s_i)` with the sign of the local field, so the product in `energy_diff` is non-positive.

The strict version, `energy_strict_descent`, adds the side condition that the update is not a no-op. Under that hypothesis, the local field and coordinate change combine to make the energy difference strictly negative.

Together these theorems establish the discrete Lyapunov property:

- every asynchronous update satisfies `E(s_{n+1}) <= E(s_n);`
- every actual state change satisfies `E(s_{n+1}) < E(s_n).`

Fixed Points as Local Energy Minima

The theorem `fixed_point_iff` connects dynamical and variational characterisations of attractors. The dynamical statement says that every asynchronous update leaves the state unchanged. The variational statement says that no single-unit flip lowers energy.

The proof uses `energy_flip_eq`, which computes the exact change in energy under a flip:

```
E(flip_i(s)) - E(s) = 2 s_i h_i(s).
```

For valid states, `s_i` is either `1` or `-1`. The sign of `s_i h_i(s)` determines whether a flip would reduce energy. Thus the local update rule and the local energy-minimality condition are equivalent.

Pigeonhole Convergence

The convergence theorem avoids continuous-time machinery entirely. No ODE existence theorem, LaSalle invariance principle, or Barbalat lemma is needed. The proof is finite and discrete.

The argument is:

1. If every update changes the state, `energy_strict_descent` gives a strictly decreasing energy value at every step.

2. If two states in the sequence were equal, their energies would also be equal, contradicting strict decrease between different times.
3. Therefore the state sequence would be injective.
4. But every valid state is a `{±1}` vector over `Fin N`, and the Boolean encoding `encodeState` places such states inside a finite type.
5. There is no injection from $\mathbb{N}$ into a finite state space.

This is the Hopfield convergence argument in its cleanest discrete form. The library formalises it as the theorem `convergence`, whose conclusion is `False` from the assumptions of an infinite state-changing asynchronous trajectory.

Standard Axioms Only

The library introduces no project-specific axioms. It is written against Lean 4 and Mathlib, uses ordinary classical reasoning where needed, and contains zero `sorry` and zero `admit`.

5. Relation to `kuramoto-lean` and `gradient-descent-lean`

`hopfield-lean` is part of the same Lean 4 formalisation programme as `kuramoto-lean` and `gradient-descent-lean`.

`kuramoto-lean` formalises finite-N Kuramoto synchronisation dynamics. Its central results are gradient identities, Lyapunov descent statements, weighted coupling theorems, Hebbian coupling identities, and related dynamical-systems facts. It studies synchronisation through phase dynamics and energy-like potentials.

`gradient-descent-lean` formalises deterministic gradient descent convergence for smooth convex optimisation. Its headline results are the standard `O(1/k)` convex convergence rate and the geometric convergence rate for smooth strongly convex objectives, stated over arbitrary real Hilbert spaces.

`hopfield-lean` complements both. Like `kuramoto-lean`, it reasons about a dynamical system using a Lyapunov function. Like `gradient-descent-lean`, it formalises a convergence mechanism central to machine learning and optimisation. Its distinctive feature is discreteness: the state space is finite, the update is asynchronous and coordinate-wise, and convergence follows from strict energy descent plus a pigeonhole argument.

Together, these libraries cover three recurring proof patterns in learning theory and AI:

- gradient descent in continuous optimisation;
- synchronisation and Lyapunov descent in coupled oscillator systems;
- attractor convergence in finite associative-memory networks.

The shared value is not merely that these results are individually checked. It is that they become importable components for larger formal arguments about learning, stability, memory, and alignment.

6. Significance for AI Safety

Hopfield networks are early models of content-addressable memory. They show how distributed local interactions can produce stable global attractors. A stored pattern is not retrieved by an explicit symbolic lookup; it emerges as the endpoint of an energy-decreasing dynamical process.

This idea remains relevant in modern AI. Contemporary work on modern Hopfield networks connects associative memory to attention mechanisms and transformer-like retrieval. Although `hopfield-lean` formalises the classical binary Hopfield model rather than modern continuous variants, the proof spine is still important: local update rules, global energy functions, fixed points, and convergence to attractors are common themes across memory models and neural computation.

For AI safety, the immediate value is foundational. Safety arguments often rely on claims about stability, convergence, attractors, or monotonic improvement of an objective. Informally, these claims can hide side conditions. A formal proof forces the side conditions into the theorem statement: finite state space, valid binary states, symmetric weights, zero diagonal, and precise update semantics.

A machine-checked Hopfield convergence theorem does not certify a deployed AI system. It does, however, provide a verified component for future formal work. It can support reasoning about simplified memory systems, attractor dynamics, discrete Lyapunov arguments, and the relationship between local update rules and global stability.

The library also provides a concrete example of how classical AI theory can be made precise in Lean 4. Many safety-relevant arguments begin as conceptual analogies: memory as attractor dynamics, learning as descent, synchronisation as coherence. Formal libraries turn those analogies into explicit mathematical artifacts with stated hypotheses and checked proofs.

7. Conclusion

`hopfield-lean` provides a compact Lean 4 / Mathlib formalisation of the classical Hopfield energy-descent and attractor-convergence argument. It defines finite binary Hopfield states, the Hopfield energy, local fields, asynchronous updates, fixed points, and Boolean state encodings. It proves the master energy-change identity, non-increase and strict decrease of energy under asynchronous updates, the fixed-point/local-minimum equivalence, and the impossibility of an infinite state-changing asynchronous trajectory.

The project is deliberately focused. It does not formalise stochastic Hopfield networks, modern continuous Hopfield networks, capacity bounds, learning rules, or transformer attention. Instead, it supplies a reliable formal core for the foundational discrete theorem. That core can now be imported and extended by future work on attractor networks, associative memory, neural computation, optimisation, and AI safety.

References

Hopfield, J. J. (1982). *Neural networks and physical systems with emergent collective computational abilities*. Proceedings of the National Academy of Sciences, 79(8), 2554-2558.

The Mathlib Community. (2024). *The Lean Mathematical Library*. GitHub repository. https://github.com/leanprover-community/mathlib4

Cassie, B. (2026). *gradient-descent-lean: Formal Proofs of Gradient Descent Convergence in Lean 4*. Zenodo. https://doi.org/10.5281/zenodo.20472996

Cassie, B. (2026). *kuramoto-lean: A Sorry-Free Lean 4 Library for Finite-N Kuramoto Synchronisation Dynamics*. Zenodo. https://doi.org/10.5281/zenodo.20468619

Cassie, B. (2026). *hopfield-lean: Lean 4 Formal Proofs of Hopfield Network Energy Descent and Attractor Convergence*. GitHub repository. https://github.com/velvetmonkey/hopfield-lean